detailed requirements

Identify Stakeholders

- **End-Users (Hikers):**
  Individuals who use the Ambulo app to plan and navigate their hiking trips.

- **Project Managers:**
  Oversee project development, ensuring milestones and objectives are met.

- **Developers:**
  Responsible for building, maintaining, and enhancing the Ambulo application.

- **Designers:**
  Create the user interface and ensure a seamless user experience.

- **System Administrators:**
  Manage backend infrastructure, databases, and ensure system reliability.

- **External Partners:**
  Providers of third-party services such as weather data APIs and mapping services.

- **Investors and Stakeholders:**
  Individuals or organizations funding the project and expecting returns or specific outcomes.

- **Regulatory Bodies:**
  Ensure the application complies with relevant laws and regulations, such as data protection standards.

Functional Requirements

1. **User Authentication and Profiles**

Profile Management:

Store user preferences, hiking history, and contribution records.
Allow users to update their profile information, including personal details and hiking preferences.

OAuth2 Integration:

Allow users to register and log in securely using their Google accounts via OAuth2.

2. **Personalized Route Planning**

Route Recommendations:

Utilize algorithms to suggest hiking routes based on user preferences, location, hiking history, and desired difficulty level.

Search and Filter:

Enable users to search for routes by

- Distance
- Region
- Loop
- Includes water sections
- Number of nights
- Trail type, Difficulty level
- Starting point
- Payment required
- Recommended season
- Surface type
- Ending point
- Estimated time
- Wheelchair Friendly
- Dog Friendly

Save and Modify Routes:

Allow users to save favorite routes to their profiles.
Enable users to modify saved routes based on changing preferences or conditions.

3. **Comprehensive Route Descriptions**

Detailed Information:
Provide comprehensive details for each route, including trail length, difficulty level, elevation gain, and estimated hiking time.

Multimedia Content:
Include photos and maps to give users a visual understanding of the route.

Points of Interest:
Highlight significant landmarks, scenic viewpoints, and rest areas along the route.

Equipment Recommendations:
Suggest necessary equipment based on route difficulty and weather conditions.

4. **Interactive Maps**

Real-Time Location Display:
Show the user's current location on the map during hikes.

Route Visualization:
Display the entire hiking route with markers for key points of interest and trail splits.

Map Layers:
Provide options to toggle different map layers such as terrain, satellite view, and trail conditions.

Zoom and Pan:
Allow users to zoom in and out and navigate around the map seamlessly.

5. **Real-Time Alerts**

Trail Condition Updates:
Provide real-time information on trail conditions, including closures, hazards, and maintenance activities.

Weather Alerts:
Notify users of sudden weather changes or extreme conditions during hikes.

6. **Weather Forecast Integration**

Weekly Forecasts:
Display detailed weekly weather forecasts to help users plan their hikes effectively.

Real-Time Weather Updates:
Integrate with external weather APIs to provide up-to-date weather information.

7. **Community Features**

User Reviews and Ratings:

Allow users to leave reviews and rate hiking routes based on their experiences in the community section.

Photo Sharing:

Enable users to upload and share photos from their hikes.

Trail Suggestions:

Let users suggest new trails and provide detailed information for inclusion in the platform.

8. **Equipment Recommendations**

Tailored Suggestions:

Recommend equipment based on the selected route’s difficulty

Customizable Lists:

Allow users to create and customize equipment lists for specific hikes.There will be an option to create a group with other users where the entire trip, including planning, equipment, and more, will be shared among them.

9. **Flexible Route Adjustments**

Dynamic Modifications:

Enable users to adjust their hiking routes in real-time based on changing preferences or unexpected conditions.

Proximity-Based Filtering:

Allow users to filter and adjust routes based on their current location.

Regional Filtering:

Provide options to filter routes by specific districts or areas of interest.

10. **Real-Time Location Features**

GPS Integration:

Utilize device GPS to track and display the user's real-time location on the map.

Context-Specific Alerts:

Provide alerts related to the user's current location, such as upcoming trail splits or areas prone to sudden weather changes.

Navigation Assistance:

Offer turn-by-turn navigation and route guidance to help users stay on track.

11. **Data Synchronization and Offline Access**

Sync Across Devices:

Ensure user data is synchronized across multiple devices for a consistent experience.

Offline Mode:

Allow users to access maps and route information offline, with data synchronization once back online.

Data Caching:

Implement caching mechanisms to store essential data locally for improved performance and reliability.

Non-Functional Requirements

1. **Performance**

Throughput:

The system should handle a minimum of 1000 concurrent users without significant performance degradation.

Resource Utilization:

Optimize server and database resources to handle high traffic efficiently.

2. **Scalability**

Horizontal Scaling:

Design the system to scale horizontally by adding more instances of each microservice to accommodate increasing user loads.

3. **Reliability and Availability**

Uptime:

Achieve an availability target of 99.9%, allowing for scheduled maintenance windows.

Redundancy:

Implement redundant servers and failover mechanisms to minimize downtime in case of component failures.

Disaster Recovery:

Establish disaster recovery plans, including regular data backups and failover strategies.

4. **Security**

Authentication and Authorization:

Implement secure OAuth2 authentication and role-based access control (RBAC) to manage user permissions.

Vulnerability Management:

Conduct regular vulnerability scanning and penetration testing using tools like

5. **Usability**

User Interface:

Design an intuitive and user-friendly interface with clear navigation and accessible design.

Accessibility:

Adhere to WCAG standards to ensure the application is accessible to users with disabilities.

Responsive Design:

Ensure the application is fully responsive and functions seamlessly across various devices and screen sizes.

6. **Maintainability**

Code Quality:

Write clean, modular, and well-documented code to facilitate easy maintenance and updates.

Documentation:

Maintain comprehensive documentation for code, APIs, and system architecture using tools like JSDoc and Swagger.

Version Control:

Use Git with GitHub for effective source code management and collaboration.

7. **Compatibility**

Browser Support:

Ensure compatibility with major browsers, including Chrome, Firefox, Safari, and Edge.

Platform Support:

Support both web and mobile platforms with consistent functionality and performance.

Device Compatibility:

Ensure the application works seamlessly across various devices, including desktops, tablets, and smartphones.

8. **Reliability**

Error Handling:

Implement robust error handling to gracefully manage and recover from system failures.

Automated Alerts:

Set up automated alerts for critical system events and performance issues to enable timely responses.

9. **Performance Optimization**

Database Optimization:

Optimize database queries and use indexing to enhance data retrieval performance.

Load Balancing:

Implement load balancing to distribute traffic evenly across servers, ensuring efficient resource utilization.

10. **Data Management**

Data Integrity:

Maintain data integrity through validation and consistent data handling practices.

Data Backup:

Perform regular data backups and implement retention policies to prevent data loss.

11. **Testing and Quality Assurance**

Automated Testing:

Implement automated testing to ensure code reliability and functionality.

System Architecture

Client-Server Architecture:

Client Side: Web and mobile applications providing user interfaces for hikers.
Server Side: Backend servers handling business logic, data processing, and API requests.
Databases: NoSQL (MongoDB) for structured data and for user-generated content.
APIs: RESTful APIs facilitating communication between client and server.

Technological Requirements

Define key technical requirements:

Browsers: Chrome, Firefox, Edge.
Devices: Desktop, tablet, mobile (Android).
Operating Systems: Windows, Linux, Android.

Frameworks: Flutter for both frontend and backend.

Infrastructure Requirements:

Hosting Platforms: AWS or Azure for backend services.
Servers: Utilize scalable cloud servers with auto-scaling capabilities.
Databases: PostgreSQL for relational data, MongoDB for NoSQL data.
Caching: shared preferences from pub.dev for in-memory caching to enhance performance.

Database Technology:

NoSQL DBMS: MongoDB for flexible data storage.
Scalability: Implement sharding and replication for database scalability.

Cloud Services:

Deployment: AWS Elastic Beanstalk or Azure App Services.
Storage: AWS S3 for user-uploaded content.
CI/CD: Jenkins for automated testing and deployment.

Development Tools:

Version Control: Git with GitHub.
IDE: Visual Studio Code.

Use Cases

Use Case 1: User Registration and Profile Management

Actor: New User

Goal: Register for an account and set up a personalized profile.

Steps:

Registration:

User navigates to the Ambulo app's registration page.
User selects "Sign up with Google" to initiate OAuth2 authentication.
Users grant necessary permissions for Ambulo to access their Google account information.
System creates a new user account and redirects the user to the profile setup page.

Profile Setup:

User input additional information such as hiking preferences, favorite trail types, and personal details.
User saves the profile information.

Possible Results:

Success: User successfully registers and sets up their profile, gaining access to personalized features.
Failure: If OAuth2 authentication fails, the system notifies the user and prompts retrying the registration process.

Use Case 2: Plan a Personalized Hiking Trip

Actor: Registered User

Goal: Create and save a personalized hiking trip based on preferences and current conditions.

Steps:

Input Preferences:

User access the "Plan a Trip" feature in the Ambulo app.
User selects preferences such as desired location, difficulty level, trail length, and preferred hiking season.

Receive Recommendations:

System processes the input using recommendation algorithms.
System displays a list of suggested hiking routes that match the user's criteria and current weather conditions and other information.

Review and Select Route:

Users review detailed descriptions, photos, and equipment recommendations for each suggested route.
User selects a preferred route and there will be two option - start, save

Save and Modify:

System saves the selected route to the user's favorite.
Users can later modify the saved route based on changing preferences or new conditions.

Possible Results:

Success: User successfully plans and saves a personalized hiking trip.
Failure: No routes match the user's criteria, prompting the user to adjust their preferences and suggesting alternative routes

---

Use Case 3: Navigate a Hike with Real-Time Alerts

Actor: Active Hiker

Goal: Navigate a hiking trail with real-time updates and alerts for safety and convenience.

Steps:

Start Navigation:

User selects a saved hiking route and clicks "Start Hike."
System activates GPS tracking and displays the current location on the interactive map.

Receive Real-Time Alerts:

As the hike progresses, the system sends real-time alerts about trail conditions, weather changes, and upcoming tricky trail splits.
Users receive push notifications and in-app alerts based on their location and predefined preferences.

Emergency Assistance:

In case of an emergency, user can access quick links to emergency contacts and safety instructions directly from the alerts.

Possible Results:

Success: User navigates the trail safely with timely alerts and assistance.
Failure: If GPS signal is lost, the system notifies the user and attempts to reconnect.

Use Case 4: Share Hiking Experience in Community

Actor: Registered User

Goal: Share reviews, photos, and suggestions to contribute to the Ambulo community.

Steps:

Create a Review:

After completing a hike, user accesses the "Community" section.
User select the completed route and write a review detailing their experience, including ratings for difficulty, scenery, and trail conditions.

Upload Photos:

User uploads photos taken during the hike to accompany the review.
Photos are tagged with location and time for better context.

Suggest a New Trail:

User selects the option to suggest a new trail if they explored an unmapped area.
User provides detailed information about the new trail, including trailhead location, length, difficulty, and points of interest.

Engage with Community:

Other users can view, like, and comment on the reviews and photos.
User receives.

Possible Results:

Success: Users successfully share their hiking experience, contributing valuable information to the community.
Failure: If the suggested trail lacks necessary details, the system prompts the user to provide additional information.
